구현 매뉴얼 · 초보자

AI Agent Pixel Office

harness가 만든 에이전트가 일하는 모습을 픽셀 사무실로 보기

harness를 바꾸지 않는다 — 옆에 시각화 어댑터를 붙인다

harness = 에이전트를 만들고 실행하는 도구.
Pixel Office = 그 에이전트들이 일하는 모습을 보여주는 화면입니다.

핵심 세 가지

1

캐릭터는 에이전트 상태를 매핑한 것이다

진짜 에이전트가 아니라 idle·running·waiting·completed·error 상태를 그림으로 보여준다.

2

harness는 건드리지 않는다

옆에 Pixel Office Adapter를 붙여 상태만 흘려보낸다.

3

한 번에 안 만들고 3단계로 쪼갠다

가짜 상태판 → harness 연결 → 픽셀 사무실 꾸미기. 각 단계가 독립적으로 돈다.

여러분이 하는 일은 이번에도 프롬프트 붙여넣기와 명령어 실행뿐이다

전체 흐름

사용자 → harness → 상태 이벤트 → 상태 저장 → Pixel Office 화면

harness = 만들고 실행 · Pixel Office = 옆에 붙는 시각화 화면



핵심: harness를 바꾸지 않는다. 상태만 흘려보내는 "Adapter"를 옆에 붙인다.

에이전트 ↔ 캐릭터 매핑

에이전트	캐릭터	상태 예시
Researcher	연구원	running · 논문 검색 중
Coder	개발자	running · 백테스트 코딩
Backtester	퀀트	waiting · 전략 대기
Reviewer	검토자	completed · 검토 완료
Slide Builder	발표자	idle

상태는 다섯 가지뿐 — idle · running · waiting · completed · error. 색·진행률·말풍선이 바뀐다.

사전 준비 — 자체 완결 설치

이미 깔린 것은 건너뛴다. 처음 한 번만.

[입력] 명령어 · 터미널 (Claude Code 설치)

```
npm install -g @anthropic-ai/claude-code
```

[입력] 명령어 · 터미널 (에이전트 팀 켜기)

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

[입력] 명령어 · Claude Code (harness 설치)

```
/plugin marketplace add revfactory/harness    →    /plugin install harness@harness-marketplace
```

Node.js·Python은 각 공식 사이트에서 설치. node/python3/claude --version 으로 확인.

1단계 — 가짜 상태판 만들기

harness 없이, 화면이 뜨는 것부터 확인한다

[입력] 프롬프트 · Claude Code

```
초보자용 AI Agent Pixel Office 데모를 만들어줘.
- 프런트: React + TypeScript + Vite. 백엔드: FastAPI.
- 상태는 처음엔 Mock 데이터로 시작(harness 아직 연결 안 함).
- 에이전트: Planner, Researcher, Strategy Extractor, Python Coder,
  Backtester, Reviewer, Slide Builder. 상태 5종에 따라 색·진행률.
- 화면엔 에이전트 카드가 격자로(사무실 맵은 나중에).
한 번에 복잡하게 말고 최소 실행 버전부터. 먼저 폴더 구조와
실행 방법을 제시하고, 내가 "다음"이라 하면 코드를 작성해라.
```

목표는 화면이 뜨는 것 하나. 아직 가짜 상태판이다.

2단계 — 실행해보기

터미널 2개를 연다

[입력] 명령어 · 터미널 1 (백엔드)

```
cd backend && pip install fastapi uvicorn && uvicorn main:app --reload --port 8000
```

[입력] 명령어 · 터미널 2 (프런트)

```
cd frontend && npm install && npm run dev
```

[출력] 브라우저 · http://localhost:5173

```
Planner       Running     Backtester   Idle
Researcher    Running     Reviewer     Completed
Python Coder  Waiting
```

3단계 — harness 에이전트 연결

.claude/agents/ 폴더를 읽어 실제 목록을 만든다

[입력] 프롬프트 · Claude Code

```
harness와 연결하는 Adapter를 만들어줘.
- .claude/agents/의 각 .md를 에이전트 하나로 등록.
- 파일 이름을 에이전트 이름으로(researcher.md
  → Researcher). 초기 상태는 idle.
- 백엔드에 GET /agents/from-harness API.
- 프런트는 이 API로 에이전트 목록을 보여준다.
```

[출력] 생성

```
harness_adapter.py

researcher.md
  → Researcher
coder.md
  → Coder
```

화면의 에이전트 목록이 실제 .claude/agents/ 기준으로 바뀐다.

상태를 실시간으로 — 두 방법

B가 진짜 자동화다

방법	원리	추천
A. 수동 JSON	Claude가 단계마다 agent_status.json을 직접 수정	맛보기용
B. 훅 자동 전송	Claude Code 훅이 상태를 백엔드로 자동 POST	실사용

방법 A의 한계

- Claude가 매번 파일을 직접 고쳐야 해서 빠뜨리기 쉽다. 진짜로 쓰려면 방법 B로 간다.

방법 B — 훅으로 자동 전송 (추천)

작업 시점마다 시스템이 상태를 자동으로 보낸다

[입력] 프롬프트 · Claude Code

```
Claude Code 훅으로 에이전트 상태를 자동 전송하게 해줘.
- .claude/settings.json에 훅 등록:
  SubagentStart→running POST, SubagentStop→completed POST,
  PostToolUse→progress 조금씩 올려 POST(선택).
- 각 훅은 이벤트 JSON(agent_type 등)을 받아
  http://localhost:8000/agents/event 로 POST 하는 작은 스크립트.
- .claude/hooks/ 아래 두고, 실패해도 작업을 막지 않게 async로.
```

왜 훅이 정답인가

- 훅은 시스템이 자동 실행한다. Claude가 상태 갱신을 빠뜨릴 일이 없다.

픽셀 사무실 꾸미기

카드형이 잘 돌면 사무실 평면도 위에 배치한다

[입력] 프롬프트 · Claude Code

```
화면을 Pixel Office 스타일로 바꿔줘.
- 배경은 사무실 평면도. 각 에이전트는 책상 위 캐릭터로.
- 고정 위치: Planner=좌상, Researcher=좌중, Extractor=중앙,
  Coder=중앙우, Backtester=하중, Reviewer=우상, Slide=우하.
- 상태에 따라 말풍선, running이면 작은 애니메이션.
- 상단 요약 배지(Total/Working/.../Progress), 우측 Activity Feed·Detail,
  하단 Workflow 단계.
```

업로드한 참고 화면처럼 배지·사무실 맵·활동 피드가 있는 대시보드가 나온다.

실전 활용 — Momentum 연구 자동화

버튼 하나로 7명이 순서대로 일한다

[입력] 프롬프트 · Claude Code

```
Momentum 연구 데모 워크플로를 추가해줘.
실행 순서: Planner → Researcher → Strategy Extractor
→ Python Coder → Backtester → Reviewer → Slide Builder.
- 처음엔 각 단계를 Mock으로 3초씩 실행(흐름부터).
- 단계마다 상태 갱신(훅 또는 agent_status.json).
- 상단에 "▶ 데모 시작" 버튼 + POST /demo/momentum/start API.
```

[출력] 화면 흐름

```
Planner: 설계
Researcher: 검색
Extractor: 팩터
Coder: 코딩
Backtester: 실행
Reviewer: 점검
Slide: 슬라이드
```

실행: 백엔드·프런트를 켜고 localhost:5173에서 ▶ 데모 시작. 익숙해지면 Researcher부터 진짜로 교체.

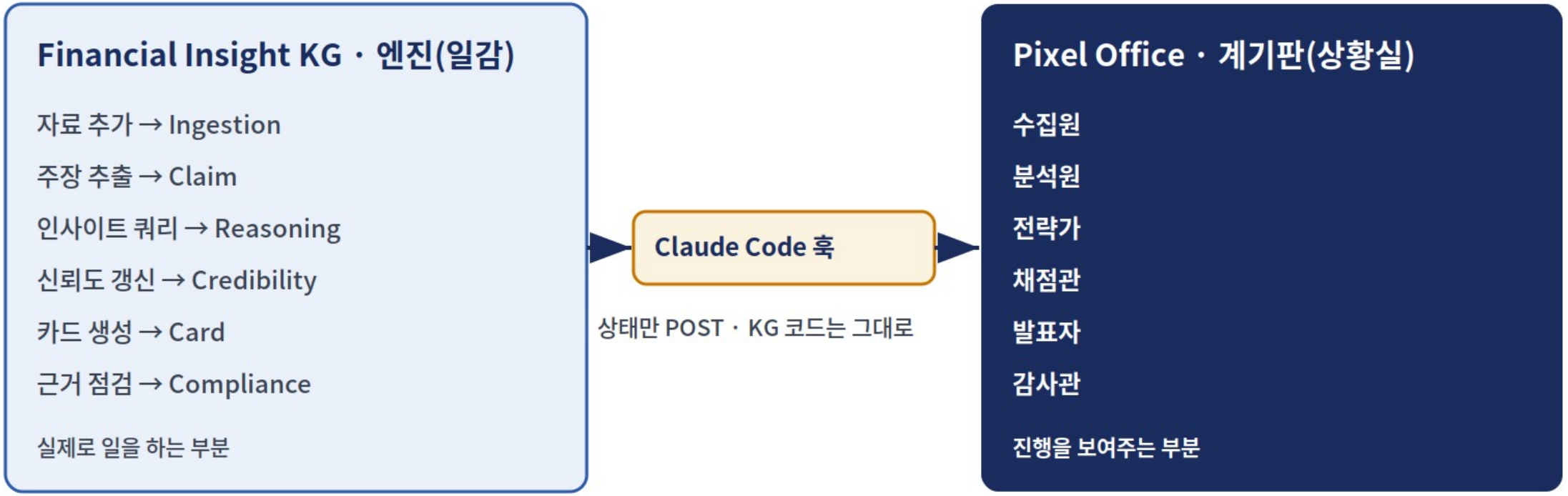
입력·출력 치트시트

단계	무엇을	어디에	형태
준비	도구·harness 설치	터미널 / Claude Code	설치 명령
상태판	Mock 데모 생성	Claude Code	프롬프트
실행	백엔드·프런트	터미널 2개	uvicorn / npm run dev
연결	harness Adapter	Claude Code	프롬프트
자동화	훅으로 상태 전송	Claude Code	프롬프트(방법 B)
활용	Momentum 데모	브라우저	▶ 시작 버튼

harness=제작 · Claude Code=실행 · 훅=상태 자동 신고 · FastAPI=서버 · React=화면.

다음 — Financial Insight KG와 연결

KG=엔진(일감), Pixel Office=계기판(상황실). 연결 고리는 Claude Code 훅 하나



KG 연결 — 프롬프트 하나

Pixel Office 백엔드를 켠 뒤 KG 프로젝트에서 붙여넣는다

[입력] 프롬프트 · Claude Code (KG 프로젝트)

```
이 Financial Insight KG를 Pixel Office 상황실 화면과 연결해줘.
- KG 에이전트(Ingestion·Claim·Reasoning·Credibility·Card·Compliance)를
  Pixel Office 캐릭터에 매핑.
- add·ask 파이프라인이 단계를 지날 때 훅(SubagentStart/Stop·PostToolUse)이
  http://localhost:8000/agents/event 로 상태를 자동 POST.
- 카드 완성→발표자 completed, Compliance 걸림→해당 캐릭터 error.
- 훅은 KG의 .claude/hooks/에 async로. Pixel Office 코드는 재사용.
```

KG를 건드리지 않고 상태만 흘려보낸다.

연결하면 이렇게 보인다 — 질문별 화면

KG에 평소처럼 묻기만 하면 사무실이 움직인다

KG 질문	사무실 화면 흐름
NVIDIA 실적 준비해줘	수집원→분석원→전략가→발표자(카드, completed)
결과 안 난 기업은?	전략가 running(그래프 탐색)→후보 completed
반도체 누가 잘 맞혀?	채점관 running(신뢰도 조회)→랭킹 completed

질문=일감 투입, 화면=처리 과정 — 결과는 KG가, 진행은 Pixel Office가 보여준다